

A Typed Named Tensor Notation for Deep Learning

A. Sepúlveda-Jiménez^{1,2}

¹Data Science-SoTE, National University

²QDR Labs

April 28, 2026

Abstract

We propose a *type discipline* for deep learning architectures in which every tensor axis carries both a name and a sort, and in which the legal operations on an axis are determined by its sort. Our central thesis is prescriptive: the most common silent bugs in deep learning code — averaging across a batch axis instead of a feature axis, dotting along a token axis instead of a key axis, applying BatchNorm to a non-i.i.d. contrastive batch — are sort errors, and a sufficiently expressive sort algebra promotes them from runtime nuisances to surface-level syntax errors. The proposal extends the named-tensor framework of [CRB23] with an algebra of axis sorts inspired by the type discipline implicit in Einstein summation notation in general relativity, where a Lorentz vector index and a spinor index cannot contract — not because such a contraction is wrong, but because the notation makes it unwriteable. We give axioms for the sort algebra; derive the legality table for contraction, broadcast, reduction, permutation, and concatenation; introduce a canonical attention glyph $\mathfrak{A}$ that exposes self vs. cross attention, masking, multi-query, grouped-query, and rotary position embedding as typed specializations of a single operator; define residual and stack combinators that make the residual stream a typed invariant of the architecture, formalizing a folk theorem of mechanistic interpretability; and exhibit, as a worked example, the type checker catching the BatchNorm-information-leak bug documented by [He+20] in contrastive self-supervised pretraining. We stress-test the resulting framework on Transformer, Vision Transformer, U-Net, Mamba, and Mixture-of-Experts architectures, and close with open problems in typed differentiation, stochastic axes, dependent shape types, and distributed-training sorts.

Keywords: named tensors; type systems; axis sorts; refinement types; equivariance; attention mechanisms; residual stream; mechanistic interpretability; Einstein notation; deep learning architectures.

1 Position

Einstein’s index summation notation does not save space. Written out, $g_{\mu\nu}V^\mu V^\nu$ is barely shorter than $\sum_{\mu,\nu} g_{\mu\nu}V^\mu V^\nu$. What it saves is *attention*. The repeated index forces the contraction to be visible; the placement (covariant vs. contravariant) forces the metric to be visible; and the convention that Lorentz indices and spinor indices belong to disjoint alphabets means that an attempted contraction between them does not type-check, even before any computation begins. The notation is, in a precise sense, a small type system.

Modern deep learning has no such notation. Practitioners read and write code in which a typical line might be `x = x.permute(0, 2, 1, 3).reshape(B, T, H * D)` — a sequence of axis manipulations whose correctness depends on a mental model of which integer indexes which semantic axis. That mental model is not encoded anywhere checkable. The most common silent bug in deep learning, in our experience and in the folklore, is reducing or contracting along

the wrong axis: averaging the loss across the feature dimension instead of the batch dimension, taking a softmax along the head axis instead of the token axis, normalizing across the wrong slice. These bugs do not crash; they merely produce models that train slowly, plateau early, or generalize poorly, and they are caught — when they are caught — by careful reading rather than by tooling.

Several lines of work have addressed pieces of the problem.

einsum and einops. The `einsum` operator, present in NumPy, PyTorch, JAX, and elsewhere, ports Einstein notation to array programming: ‘`bthd,bThd->bthT`’ means contract along d , broadcast along b, h , and produce a result with the named indices. The expressive payoff is real, and `einops` [Rog22] extends this with reshape and reorder operations under named axes. But neither system has a notion of *what kind of axis* an index ranges over: the letter `b` is no different from the letter `d` to the type checker. A user can `einsum` along the batch axis without complaint.

Named tensor notation. The closest predecessor to this work is [CRB23], building on the labelled-axis tradition of `xarray` [HH17] and Dex [Pas+21]. Tensors carry named axes rather than positional ones; contraction is by name match; broadcasting is unambiguous. The framework is rich enough to write a Transformer in a page. It does not, however, distinguish *kinds* of axes: a name is a name. The named-tensor framework is therefore a necessary substrate for what we propose, but not sufficient.

Penrose / tensor network diagrams. Used heavily in many-body physics — matrix product states, MERA [BB17] — and ported to machine learning by [SS16], tensor diagrams make contraction structure visually explicit. They have two limitations for our purpose. First, they are inherently linear: a softmax or a layer norm appears as an opaque box, breaking the diagrammatic discipline at exactly the points where modern architectures live. Second, they are typeless in the same sense as `einsum`.

String diagrams from categorical machine learning. A more ambitious line — [FST19] on backpropagation as a functor; [Cru+22] on gradient categories; [Cru+24] on parametric lenses; the DisCoPy framework [FTC20] — gives backpropagation a categorical semantics in which each layer is a morphism in a particular monoidal category. The framework is mathematically beautiful and handles differentiation natively. Its limitation is ergonomic: a Transformer block written as a string diagram is currently not readable by working ML researchers without significant translation, and the categorical machinery is heavier than a typical paper requires.

Dependent shape types. Dex [Pas+21] encodes tensor shapes in a dependently typed system that can statically verify shape compatibility including dynamic dimensions like sequence length. This is the most expressive route; it is also the heaviest, and the encoding overhead is currently incompatible with paper notation.

Mechanistic interpretability. The interpretability literature has independently arrived at the conclusion that the residual stream of a Transformer is the right unit of analysis. [Elh+21] introduce the framing of attention heads as readers and writers on a shared residual stream; [Ols+22] use this framing to identify induction heads as a circuit-level mechanism. Activation patching [Men+22], automated circuit discovery [Con+23], and dictionary-learning interpretability [Bri+23] all rely on the residual stream having a stable type across depth. None of this work gives the residual stream a formal type, but the use of the construct in proofs — “patch the residual stream at layer ℓ_1 from one input into the forward pass on another” — is implicitly typed reasoning. A formal account is one of the contributions of this paper (Section 5).

The gap, stated as a criterion. The gap is therefore well-defined, and we adopt it as a falsifiable criterion: a successful framework must (a) be readable by working ML researchers without retraining; (b) be sort-disciplined in the Einstein sense, so that named common axis-confusion bugs become syntax errors rather than runtime errors; and (c) be compositional, with first-class symbolic forms for residual stacks, attention variants, and architectural patterns. A framework that meets (a) and (c) but not (b) is a notation; one that meets (b) and (c) but not (a) is a type system without users. We aim for all three. Sections 2 through 5 develop the framework. Section 3.4 demonstrates criterion (b) on a documented bug from the contrastive-learning literature [He+20]. Section 6 stress-tests the framework on five architectures. Section 7 lists what remains to be done.

2 The axis sort algebra

We begin by carving up the universe of tensor axes into sorts. The choice of sorts is empirical: a sort is justified if it predicts a difference in legal operations.

2.1 Sorts

We propose the set

$$\mathcal{S} = \{\mathbf{B}, \mathbf{T}, \mathbf{F}, \mathbf{H}, \mathbf{K}, \mathbf{V}, \mathbf{C}, \mathbf{S}, \mathbf{L}, \mathbf{G}, \mathbf{N}, \mathbf{M}\} \quad (1)$$

denoting batch (**B**), token or time (**T**), feature (**F**), head (**H**), key (**K**), vocabulary (**V**), channel (**C**), spatial (**S**), layer (**L**), group (**G**), noise (**N**), and memory (**M**). The first ten cover everything in a standard Transformer, ViT, or convolutional network; the last two are required by stochastic and recurrent architectures and we treat them as first-class.

2.2 Operations

We consider eight elementary operations on tensors. Each is parametrized by the axis or axes on which it acts.

- Bcast_a : broadcast along axis a .
- Sum_a , Mean_a , Max_a : reductions along a .
- $\text{Contract}_{a,\mathcal{O}}$: contract a pair of axes named a under operator $\mathcal{O}$.
- $\text{Perm}_{a,b}$: permute two axes (only well-defined within a sort that admits permutation).
- Concat_a : concatenate two tensors along a to extend its dimension.
- Slice_a : select a contiguous sub-range along a .
- Index_a : project to a single value along a .
- Norm_a : parametrized normalization (mean-and-rescale) along a .

2.3 Axioms

The legality of each operation on each sort is governed by the following axioms.

Axiom 1 (Sample independence). Axes of sort **B** admit broadcasting and reduction only via a designated *loss reduction operator* $\mathcal{L}$. They admit no contraction, no concatenation with non-**B** axes, and no permutation that mixes them with non-**B** axes.

The rationale is that the batch axis indexes statistically independent samples; any operation that breaks this independence destroys the meaning of the tensor. This is the typed expression of the standard assumption that gradient estimates are unbiased over the batch.

Axiom 2 (Order preservation). Axes of sort **T** and **S** carry an ordering. Operations that depend on the ordering — causal masking, position-encoded attention [Su+24], convolution kernels — must respect it; operations that ignore it (set-style pooling, permutation-invariant aggregation) must be explicitly declared.

Axiom 3 (Licensed contraction). Each sort declares a finite set of *licensed* contraction operators. Sort **F** licenses the dot product (linear layers). Sort **K** licenses the dot product *only inside attention*. Sort **T** licenses contraction *only via attention or convolution*. Sort **S** licenses contraction via convolution and pooling. Sort **H** licenses head-mixing (typically via concatenation followed by a linear projection). Sorts **B**, **V**, **N** license no contraction.

Axiom 4 (Reduction restriction). Sort **V** admits only the reductions LogSumExp_v and Argmax_v (and their negations, softmax, and sampling forms). Sort **B** admits only the loss reduction $\mathcal{L}$. Sort **N** admits only the marginalization $\mathbb{E}_n$, which is the licensed loss-time operation; reducing **N** before the marginalization step is illegal.

The rationale is that logits are not summed by counting, batch is not averaged before the loss, and noise is not collapsed before its statistical role is discharged.

Axiom 5 (Identity broadcast). Every sort admits identity broadcast: $\text{Bcast}_a(X)$ is legal whenever a is not already present in X ’s signature. Broadcasting introduces no semantic change beyond axis introduction.

Axiom 6 (Concatenation closure). Concat_a is legal iff the two operands agree on every axis except a , *and* the sort of a permits concatenation. Sorts **F**, **C**, **H**, **V** permit concatenation. Sort **B** permits concatenation only if both operands come from the same sample distribution (a side condition, not a syntactic check). Sorts **T**, **S** permit concatenation only when it preserves the ordering; in particular, two sequences of length $|t_1|$ and $|t_2|$ concatenate to a sequence of length $|t_1| + |t_2|$, in that order.

Axiom 7 (Equivariance). For each sort $s \in \mathcal{S}$, fix a designated subgroup $G_s \subseteq S_n$ acting on each axis $a : s$ with $|a| = n$ by index permutation: for $\sigma \in G_s$, define

$$(\pi_a^\sigma X)[\dots, a=i, \dots] = X[\dots, a=\sigma(i), \dots].$$

Every operation O acting on a sort- s axis a must satisfy:

- (a) if O preserves a in its output signature, then O is G_s -equivariant: $O(\pi_a^\sigma X) = \pi_a^\sigma O(X)$ for all $\sigma \in G_s$;
- (b) if O eliminates a from its output (a reduction or contraction), then O is G_s -invariant: $O(\pi_a^\sigma X) = O(X)$ for all $\sigma \in G_s$.

The groups G_s are specified by the sort:

- $G_{\mathbf{B}} = G_{\mathbf{H}} = G_{\mathbf{G}} = G_{\mathbf{N}} = S_n$ (full symmetric group): batch elements, attention heads, MoE experts, and noise samples are exchangeable.
- $G_{\mathbf{T}} = G_{\mathbf{S}} = G_{\mathbf{M}} = \{e\}$ (trivial): position, spatial location, and recurrent state ordering all matter.
- $G_{\mathbf{F}} = G_{\mathbf{K}} = G_{\mathbf{V}} = G_{\mathbf{C}} = \text{matched}$: a permutation along a feature, key, vocabulary, or channel axis is legal only if matched by the corresponding permutation of every weight that contracts with that axis. (Equivalently: these axes carry semantics only up to a relabeling that is a symmetry of the parameters.)
- $G_{\mathbf{L}} = S_n$ when the family $\{f_\ell\}$ is tied (all $f_\ell = f$); $G_{\mathbf{L}} = \{e\}$ otherwise.

Remark 8 (Consequence: loss reductions must be symmetric). Theorem 7 together with Theorem 1 implies that the loss reduction $\mathcal{L}$ is necessarily a symmetric function of the batch: $\mathcal{L}(\pi_b^\sigma X) = \mathcal{L}(X)$ for all $\sigma \in S_{|b|}$. This is the typed expression of the standard assumption that a per-sample loss followed by mean-aggregation is the canonical training objective; it rules out, e.g., a “loss” that depends on the order of samples within a minibatch.

2.4 The legality table, derived

From Theorems 1 to 7 we derive the legality table below. An entry “✓” indicates the operation is legal under any conditions consistent with the operation’s normal use; “†” indicates legal only with a side condition stated in the axioms; “×” indicates illegal.

Sort	Bcast	Sum/Mean	Max	Contract	Perm	Concat	Slice
B batch	✓	† ($\mathcal{L}$ only)	×	×	† (within B)	†	✓
T token	×	† (pooling)	† (pooling)	† (attn/conv)	×	✓	✓
F feature	✓	✓	✓	✓ (dot)	✓	✓	✓
H head	×	† (head-mean)	×	† (concat-project)	✓	✓	✓
K key	×	✓	×	† (in $\mathfrak{A}$ only)	✓	✓	✓
V vocab	×	† (LSE only)	† (argmax)	×	×	✓	✓
C channel	✓	✓	✓	✓ (1×1)	✓	✓	✓
S spatial	†	† (pool)	† (pool)	† (conv)	×	✓	✓
L layer	×	† (deep ens.)	×	×	† (when tied)	✓	✓
G group	✓	† (gating)	† (top- k)	† (gated mix)	✓	✓	✓
N noise	✓	† ($\mathbb{E}_n$ only)	×	×	✓	✓	✓
M memory	×	† (state-readout)	×	† (recurrence)	×	†	✓

Three observations on the table.

First, the feature sort **F** is the most permissive: it admits every operation. This is correct: features are the “free” axis of deep learning, the dimension along which the model has learned to put whatever it pleases. Every other sort is restricted relative to **F**, and the restrictions encode semantic content.

Second, the table is asymmetric in a way that matches practice. Sort **B** is nearly fully restricted, because the batch axis is a statistical scaffold, not a representational axis. Sort **T** is restricted in interesting ways: it is the sort that licenses attention, but only attention. A user writing `X.sum(dim=t)` on a non-pooling layer is committing a sort error.

Third, several entries that practitioners might intuitively call legal are marked †, meaning legal-with-conditions. These are exactly the places where deep learning bugs cluster: head-axis reduction (legal as final mean across heads, illegal in the middle of a block), layer-axis reduction (legal in deep ensembles, illegal in standard stacks), batch concatenation (legal in data loading, illegal in the middle of training).

2.5 Refinement subsorts

In some practical settings the basic sorts of Section 2 are too coarse: a **B**-axis carrying paired views (as we will see in Section 3.4) and a **B**-axis carrying i.i.d. samples both lie in sort **B**, but admit different operations. We extend the algebra with a *refinement order* that allows narrowing without widening.

Definition 9 (Refinement). A sort s' is a *refinement* of sort s , written $s' \sqsubseteq s$, if every legality restriction stated for s in Theorems 1 to 7 also holds for s' . The refinement may add additional restrictions: an operation O on a sort- s' axis is legal only if O is legal on a sort- s axis *and* legal under any additional axioms attached to s' . The relation $\sqsubseteq$ is reflexive, transitive, and makes $\mathcal{S}$ a meet-semilattice with each basic sort the top of its refinement chain.

Proposition 10 (Operational consequences of refinement). *For any refinement $s' \sqsubseteq s$:*

- (i) *Implicit upcast. A tensor with axis $a : s'$ is implicitly usable wherever a tensor with axis $a : s$ is required, but not vice versa.*
- (ii) *Group restriction. The equivariance group satisfies $G_{s'} \subseteq G_s$: refinement can only narrow exchangeability, never extend it.*

- (iii) Licensure narrowing. *If an operation O is licensed on sort- s axes but not on sort- s' axes, then O on a sort- s' axis is a type error; the converse direction cannot occur.*

Proof sketch. (i) follows from the inclusion of legality restrictions: any expression well-typed at sort s remains well-typed when the input is at the more restricted sort s' . (ii) follows from Theorem 7: refinement must respect the equivariance dichotomy, and a wider $G_{s'}$ would license operations that G_s forbids. (iii) is the contrapositive of the legality narrowing implied by the additional axioms. $\square$

Example 11 (Refinements used in this paper). The following refinements appear in subsequent sections; we record them here for reference.

- $\mathbf{B}^{\text{iid}} \sqsubseteq \mathbf{B}$: full exchangeability ($G = S_n$); admits BatchNorm-style parametric reductions. Used in Section 3.4.
- $\mathbf{B}^{\text{paired}} \sqsubseteq \mathbf{B}$: paired views with $G = S_{n/k} \wr S_k$ (the wreath product reflecting that groups of k paired samples are exchangeable as units, but samples within a group are not interchangeable with samples across groups in the way independent samples would be); rejects parametric reductions across pairs. Used in Section 3.4.
- $\mathbf{B}^{\text{seq}} \sqsubseteq \mathbf{B}$: temporally correlated samples (e.g., RL rollouts) with $G_{\mathbf{B}^{\text{seq}}} = \{e\}$; rejects all reductions except $\mathcal{L}$. Used in Section 7.
- $\mathbf{T}^{\text{causal}} \sqsubseteq \mathbf{T}$: token axis with declared causal structure; admits attention only with causal masks. Anti-causal contraction is a type error.
- $\mathbf{T}^{\text{set}} \sqsubseteq \mathbf{T}$: token axis declared as an unordered set; recovers $G_{\mathbf{T}^{\text{set}}} = S_n$ from the parent’s trivial group, in exchange for losing position-encoded operations.
- $\mathbf{N}^{\text{iid}} \sqsubseteq \mathbf{N}$ and $\mathbf{N}^\pi \sqsubseteq \mathbf{N}$: i.i.d. noise vs. policy-conditioned noise (Section 7.2).

Remark 12 (Decidability under refinement). Since the refinement lattice is finite (in any concrete document, the user declares a finite set of refinements), refinement checking is constant-time. The type-checking algorithm of Theorem 16 therefore remains linear in the size of the expression tree under the extended algebra.

3 Typed named tensors

We now equip the sort algebra with the named-tensor framework of [CRB23].

3.1 Definitions

Definition 13 (Sorted axis). A *sorted axis* is a triple $a = (n_a, s_a, |a|)$ where n_a is a name (drawn from a fixed alphabet), $s_a \in \mathcal{S}$ is a sort, and $|a| \in \mathbb{N}$ is a dimension. We write $a : s$ for “axis a has sort s .”

Definition 14 (Tensor signature). A *signature* is a finite set of sorted axes, all with distinct names, written $\Sigma = \{a_1, \dots, a_n\}$.

Definition 15 (Typed named tensor). A *typed named tensor* with signature Σ is a function

$$X : \prod_{a \in \Sigma} [|a|] \longrightarrow \mathbb{R},$$

notated $X^{[\Sigma]}$ or, when the signature is clear, $X^{[a_1, \dots, a_n]}$.

The order of axes in the superscript is irrelevant: $X^{[b, t, d]}$ and $X^{[t, d, b]}$ denote the same object. This is the central convenience of named-tensor notation.

3.2 Operations

Contraction. Given $X^{[\Sigma_1]}$ and $Y^{[\Sigma_2]}$ with $a \in \Sigma_1 \cap \Sigma_2$, the contraction along a is

$$(X \cdot Y)^{[\Sigma_1 \Delta \Sigma_2]} = \sum_a X^{[\Sigma_1]} Y^{[\Sigma_2]}, \quad (2)$$

provided a ’s sort licenses contraction in the current operator context (Theorem 3). The result has signature $\Sigma_1 \Delta \Sigma_2$ (symmetric difference). When multiple shared axes are present, all of them are contracted simultaneously, again subject to license. Names appearing in only one operand are broadcast.

Broadcast. A name $a \in \Sigma_1 \setminus \Sigma_2$ in a binary operation is broadcast through the result iff a ’s sort permits broadcasting (Theorem 5 together with the legality table). Otherwise the operation is ill-typed.

Reduction. $\text{Sum}_a, \text{Mean}_a, \text{Max}_a$ are unary operators that remove a from the signature:

$$(\text{Sum}_a X)^{[\Sigma \setminus \{a\}]} = \sum_a X^{[\Sigma]}. \quad (3)$$

Each reduction is legal iff the sort of a permits it.

Pointwise operations. For a function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$, the pointwise lift is

$$\sigma \langle X^{[\Sigma]} \rangle^{[\Sigma]} = \sigma(X^{[\Sigma]}), \quad (4)$$

applied entrywise. The angle brackets are the syntactic marker for “no contraction, all axes pass through.” This is what tensor diagrams call an “opaque box” but typed.

Concatenation. $\text{Concat}_a(X, Y)$ is defined when X and Y agree on $\Sigma \setminus \{a\}$ and disagree on a only in dimension; the result has $|a|_{\text{new}} = |a|_X + |a|_Y$. Legal iff a ’s sort permits concatenation (Theorem 6).

Normalization. For a a feature-like axis,

$$\text{Norm}_a \langle X^{[\Sigma]} \rangle = \gamma^{[a]} \odot \frac{X^{[\Sigma]} - \mu_a^{[\Sigma \setminus \{a\}]}}{\sigma_a^{[\Sigma \setminus \{a\}]}} + \beta^{[a]}, \quad (5)$$

where μ_a, σ_a are the mean and standard deviation reductions along a . The normalization axis is part of the operator’s name, which kills the BatchNorm [IS15] / LayerNorm [BKH16] / RMSNorm [ZS19] confusion at the syntactic level: Norm_b is BatchNorm, Norm_d is LayerNorm, Norm_s is InstanceNorm, and they are not interchangeable.

3.3 Type checking

Proposition 16 (Decidability of type checking). *Given an expression tree built from the operations of § 3.2, computing the resulting signature and verifying sort legality is decidable in time linear in the size of the expression tree.*

Proof sketch. The algorithm proceeds bottom-up. At each node:

1. Compute the signatures of the children.
2. Determine the operation’s sort requirements.

3. Verify that every shared name has a sort permitting the requested operation (per Theorems 1 to 7).
4. Verify that every non-shared name has a sort permitting broadcasting through the operation.
5. Compute the output signature.

A type error is raised on any axiom violation. Each step is constant time given the finite sort lattice and first-order legality predicates; the algorithm is therefore linear in the expression size. $\square$

Example 17 (A type error). The expression $\text{Mean}_b(W^{[d,d']}X^{[b,t,d]})$ raises a sort error at the outer node, because Mean_b on a $\mathbf{B}$ -axis is illegal except via $\mathcal{L}$ (Theorem 4). The user must instead write $\mathcal{L}[W^{[d,d']}X^{[b,t,d]}]$, which forces them to declare that this is a loss computation.

3.4 A worked example: catching a documented bug

We illustrate the type checker on a documented sort error from the contrastive-learning literature. [He+20] report, in §3.3 of the MoCo paper, that applying standard BatchNorm [IS15] during contrastive self-supervised pretraining produces a systematic information leak: the model can solve the contrastive task by exploiting batch statistics rather than learning content-level features. Their fix — *Shuffle BN*, distributing paired views across GPUs so that each per-device sub-batch is i.i.d. again — has become standard [He+20; Che+20; Car+20]. We show that this bug is precisely a sort error in our framework, and that the canonical fix is a typed sort-changing operator.

Setup. Refine the batch sort $\mathbf{B}$ into two subsorts that differ in their group $G_{\mathbf{B}}$ (Theorem 7):

- $\mathbf{B}^{\text{iid}}$: batches of statistically independent samples; $G_{\mathbf{B}^{\text{iid}}} = S_n$. Reductions along $\mathbf{B}^{\text{iid}}$ are licensed (per Theorems 1 and 4) for any symmetric reduction, including the BatchNorm statistics.
- $\mathbf{B}^{\text{paired}}$: batches in which samples come in correlated groups (e.g. two augmented views of each image); $G_{\mathbf{B}^{\text{paired}}} = S_{n/k} \wr S_k$, the wreath product reflecting that groups of k paired samples are exchangeable as units, but samples within a group are not interchangeable with samples in other groups in the way independent samples would be. Crucially, no symmetric reduction over $\mathbf{B}^{\text{paired}}$ commutes with the loss in the way Theorem 8 requires.

A contrastive pretraining batch has signature

$$X^{[b : \mathbf{B}^{\text{paired}}, c : \mathbf{C}, s_1 : \mathbf{S}, s_2 : \mathbf{S}]}.$$

The buggy expression. A user, copy-pasting an encoder from a supervised pipeline, writes

$$X' = \text{Norm}_b(X).$$

Type-checker trace. The bottom-up algorithm of Theorem 16 proceeds as follows.

1. Read signature of X : contains axis $b : \mathbf{B}^{\text{paired}}$.
2. Identify outer operator: Norm_b requires the reductions μ_b, σ_b along b .
3. Check legality of Mean_b on a $\mathbf{B}^{\text{paired}}$ -axis: by Theorem 1 and Theorem 4, $\mathbf{B}$ -axis reductions are licensed only via $\mathcal{L}$ or via parametric normalization on a $\mathbf{B}^{\text{iid}}$ -refined axis.
4. $\mathbf{B}^{\text{paired}} \not\subseteq \mathbf{B}^{\text{iid}}$ by declaration.
5. **Type error:** Norm_b is illegal on a $\mathbf{B}^{\text{paired}}$ -axis.

The corrected expressions. The type system accepts at least three principled fixes, each corresponding to a real practice in the literature:

- (i) $X' = \text{Norm}_d(X)$ (LayerNorm [BKH16]): legal because $d : \mathbf{F}$ admits unrestricted reduction; this is the route taken by SimCLR-style models that use MLP projection heads with layer-norm.
- (ii) $X' = \text{Norm}_b(\text{ShuffleBN}_{b \rightarrow \mathbf{D}}(X))$, where ShuffleBN is declared as a sort-changing operator $\mathbf{B}^{\text{paired}} \rightarrow \mathbf{B}^{\text{iid}}$ that distributes paired samples across the device sort $\mathbf{D}$ (§ 7.4) so each per-device sub-batch is i.i.d.; this is the original fix of [He+20].
- (iii) Reshape: split $b : \mathbf{B}^{\text{paired}}$ into $b' : \mathbf{B}^{\text{iid}}$ and $v : \mathbf{G}_2$ (a group axis indexing the two views), then apply $\text{Norm}_{b'}$ rather than Norm_b . The resulting normalization sees only one view per pair and respects independence; this is the route taken by some SimCLR re-implementations.

Discussion. An information-leak bug that took months of experimental analysis to diagnose in 2019–2020 [He+20] becomes, in the typed framework, a syntactic error caught at the type-checking step, with three canonical fixes mechanically suggested by the constraint that needs to be discharged. We do not claim that the type system catches every contrastive-learning bug — some require run-time checks of statistical properties that no syntactic system can verify. We do claim that the *axis-confusion* component of such bugs, which is the most common failure mode and the one most resistant to detection by testing, becomes unwriteable. This is the prescriptive sense in which the framework is a type system rather than merely a notation: it converts a class of latent runtime bugs into surface-level syntax errors.

4 The attention glyph

Attention [Vas+17] is the dominant primitive of the past decade and warrants a dedicated symbol. We define

$$\boxed{\mathfrak{A}_{k,v}^{h;t \leftarrow t'}[Q, K, V \mid M]} \quad (6)$$

with the following reading. The superscripts and subscripts declare the sort signature of the attention; the bracketed arguments are the tensors.

- $h : \mathbf{H}$ is the head axis, carried through in parallel.
- $t : \mathbf{T}$ is the *query* token axis (the axis of the output).
- $t' : \mathbf{T}$ is the *key/value* token axis (the axis being summarized).
- $k : \mathbf{K}$ is the key dimension, contracted by the $QK^\top$ product.
- $v : \mathbf{F}$ is the value (output feature) dimension, the head-local feature axis after attention.
- M is an optional mask of signature $[t, t']$.

The operator is defined by

$$\mathfrak{A}_{k,v}^{h;t \leftarrow t'}[Q, K, V \mid M] = \text{softmax}_{t'} \left\langle \frac{1}{\sqrt{|k|}} Q^{[b,t,h,k]} K^{[b,t',h,k]} + M^{[t,t']} \right\rangle V^{[b,t',h,v]}, \quad (7)$$

where the implicit contractions over k and t' are forced by name-matching, the head axis h and batch axis b pass through by the sort algebra, and the softmax-along- t' is the licensed reduction operator.

4.1 Variants as typed specializations

The point of the glyph is that every attention variant is a typed modification of $\mathfrak{A}$, expressible by changing the signature rather than rewriting the body.

Self vs. cross attention. If $t = t'$ (same name, same sort), it is self-attention. If $t \neq t'$, cross attention. Visible in the glyph; no code change needed.

Causal masking. Define $M_{\text{causal}}^{[t,t']} = -\infty \cdot \mathbb{K}[t' > t]$. Causal self-attention is $\mathfrak{A}_{k,v}^{h;t \leftarrow t'}[Q, K, V \mid M_{\text{causal}}]$.

Multi-query attention [Sha19]. Drop h from K and V :

$$\mathfrak{A}_{k,v}^{h;t \leftarrow t'}[Q^{[b,t,h,k]}, K^{[b,t',k]}, V^{[b,t',v]}].$$

The notation makes the omission visible. The head axis broadcasts over the missing factors per Theorem 5 — but only because the sort algebra licenses head-broadcasting.

Grouped-query attention [Ain+23]. Introduce a group axis $g : \mathbf{G}$ with h factoring as $h = g \otimes h'$:

$$K^{[b,t',g,k]}, V^{[b,t',g,v]}, Q^{[b,t,g,h',k]},$$

broadcasting over h' . One symbol-level change.

Rotary position embedding (RoPE) [Su+24]. RoPE is a position-conditioned linear modification of Q and K :

$$\tilde{Q}^{[b,t,h,k]} = R(t) Q^{[b,t,h,k]}, \quad \tilde{K}^{[b,t',h,k]} = R(t') K^{[b,t',h,k]},$$

where $R(\cdot)$ is a block-diagonal rotation acting on k . Substitute $\tilde{Q}, \tilde{K}$ for Q, K in $\mathfrak{A}$. The fact that RoPE acts inside $\mathfrak{A}$ rather than outside is now a typed statement: the rotation is on the $\mathbf{K}$ -axis, and is therefore licensed only inside the attention operator that licenses $\mathbf{K}$ -contraction.

Sliding-window attention. Replace M_{causal} with $M_{\text{window}}^{[t,t']} = -\infty \cdot \mathbb{K}[|t - t'| > w]$. No other change.

Linear attention [Kat+20]. Replace softmax-along- t' with a kernel feature map ϕ followed by associative regrouping:

$$\mathfrak{A}_{\text{lin}}^{h;t \leftarrow t'}_{k,v}[Q, K, V] = \phi(Q)^{[b,t,h,k']} (\phi(K)^{[b,t',h,k']} V^{[b,t',h,v]}), \quad (8)$$

where k' is a feature-map dimension and the t' -contraction now happens inside the inner product rather than as a softmax reduction. The associativity of the typed contraction makes the $O(|t|)$ vs. $O(|t|^2)$ distinction immediate.

KV-caching. Caching is the statement that K, V are partially precomputed across t' . In the notation, this is a partial application of $\mathfrak{A}$ along t' : split $t' = t'_{\text{cache}} \sqcup t'_{\text{new}}$ via concatenation, and recompute only the t'_{new} slice. Because $\mathbf{T}$ -concatenation is licensed by Theorem 6 with order preservation, this is a sort-clean operation. [Dao+22] reorganize this contraction for hardware efficiency without changing its typed structure.

The takeaway: the seven major attention variants of the past five years are all single-symbol modifications of $\mathfrak{A}$. Other recent variants such as Multi-head Latent Attention [Dee24] fit the same pattern with a low-rank factorization on the k, v axes.

5 Residual and stack combinators

The compositional structure of deep architectures [He+16] is dominated by two patterns: residual connections and layer stacking. We give each a combinator.

5.1 Residual lift

Definition 18 (Residual lift). Let $f : X^{[\Sigma]} \mapsto Y^{[\Sigma']}$ be an operator. The *residual lift* of f is defined when $\Sigma' = \Sigma$ — that is, when f preserves the entire signature including all sorts and names — and is given by

$$\mathcal{R}[f] = \text{id} + f. \quad (9)$$

The signature-preservation requirement is the typed content of “residual stream”. The operator f may compute anything internally, but its output must inhabit the same typed slot as its input. A linear projection $W^{[d,d']}$ that maps $\mathbf{F}$ -dimension d to a different $\mathbf{F}$ -dimension d' is not lift-able by $\mathcal{R}$ — there is no canonical residual connection across dimension changes, exactly as in practice.

5.2 Stack combinator

Definition 19 (Stack). For a family $\{f_\ell\}_{\ell=1}^L$ of operators each preserving signature Σ , the *stack* is

$$\bigodot_{\ell=1}^L f_\ell = f_L \circ f_{L-1} \circ \cdots \circ f_1. \quad (10)$$

The stack itself preserves Σ . The layer index $\ell : \mathbf{L}$ is available for indexing, tying ($f_\ell = f$ for all ℓ , as in ALBERT [Lan+20]), or scheduled variation (f_ℓ depends explicitly on ℓ , as in stochastic depth [Hua+16]).

5.3 The residual stream is a typed invariant

Theorem 20 (Residual stream invariance). *Let Σ be a signature and let $\{g_\ell\}_{\ell=1}^L$ be a family of operators each of the form $g_\ell = \mathcal{R}[f_\ell]$ for some Σ -preserving f_ℓ . Define the partial stack $S_k = \bigodot_{\ell=1}^k g_\ell$ for $0 \leq k \leq L$, with $S_0 = \text{id}$. Then for every k , S_k preserves Σ .*

Proof. By induction on k . The base case $k = 0$ is immediate since $S_0 = \text{id}$. For the inductive step, $S_k = g_k \circ S_{k-1}$. By the inductive hypothesis S_{k-1} preserves Σ , and $g_k = \text{id} + f_k$ preserves Σ by Theorem 18. The composition of signature-preserving operators preserves signatures. $\square$

The theorem is mathematically trivial. Its content is that we have *named* the invariant: there is a typed object — the residual stream, with signature Σ — that every layer of a residual stack acts on without changing its type. This formalizes the observation, central to the interpretability literature [Elh+21; Ols+22], that the residual stream is the right unit of analysis in a transformer.

We extract three corollaries, each of which is a folk theorem in the interpretability literature now made formal.

Corollary 21 (Logit lens). *For every k , the composition $U \circ \text{Norm}_d \circ S_k$ is a well-typed map from the input to the vocabulary signature $[b, t, v]$.*

That is: the logit lens of [nos20] — applying the unembedding to intermediate residual stream states — is type-safe at every layer because the residual stream type does not change with depth. In a non-residual architecture this would not type-check.

Corollary 22 (Linear probes). *For every k , a linear probe $P^{[d,p]}$ of any feature-axis dimension d and probe dimension p is well-typed at layer k .*

Corollary 23 (Activation patching). *For any k_1, k_2 and any pair of inputs x_1, x_2 , the patched computation that replaces the k_1 -th residual stream activation in the forward pass on x_1 with the k_1 -th residual stream activation from x_2 is type-safe.*

Activation patching [Men+22; Con+23], the workhorse of mechanistic interpretability, requires that the patched activations have the same type as those they replace. Theorem 20 guarantees this by construction.

A transformer block, in this notation, is

$$\text{TB}_\ell = \mathcal{R}[\text{MLP}_\ell \circ \text{Norm}_d] \circ \mathcal{R}[\mathfrak{A}_\ell \circ \text{Norm}_d], \quad (11)$$

with pre-LayerNorm placement made syntactic by the position of Norm_d inside $\mathcal{R}$. Post-LN places it outside. Different normalization placements (sandwich norm, peri-LN) become symbol-level rearrangements.

6 Case studies

We stress-test the framework on five architectures of varying structure.

6.1 GPT-style decoder Transformer

Declare $b : \mathbf{B}, t : \mathbf{T}, d : \mathbf{F}, h : \mathbf{H}, k : \mathbf{K}, v : \mathbf{V}, \ell : \mathbf{L}$. The forward pass is

$$y^{[b,t,v]} = U^{[d,v]} \text{Norm}_d \left\langle \bigodot_{\ell=1}^L \text{TB}_\ell \left(E^{[v,d]} x^{[b,t,v]} + P^{[t,d]} \right) \right\rangle, \quad (12)$$

where E is the embedding, P the positional encoding (or $P = 0$ if RoPE is folded into the attention), TB_ℓ the transformer block of Equation (11), and U the unembedding (which may share storage with E via the constraint $U = E^\top$).

Every contraction is forced by name. Every reduction is sort-licensed. There is no axis-permutation bookkeeping. Switching to MQA or GQA changes only the sort signatures of $\mathfrak{A}_\ell$.

6.2 Vision Transformer

ViT [Dos+21] introduces a *patchification* step that converts spatial axes to a token axis. In typed notation, this is a sort-changing operation declared explicitly:

$$\text{Patch} : X^{[b,c,s_1,s_2]} \mapsto X'^{[b,t,d]}, \quad (13)$$

where the new token axis $t : \mathbf{T}$ has dimension $|t| = (|s_1|/p)(|s_2|/p)$ for patch size p , and the new feature axis $d : \mathbf{F}$ has dimension $|d| = p^2 \cdot |c|$. The sort change is the typed content of patchification: spatial structure is relinquished (sort $\mathbf{S}$ has order-preserving constraints; $\mathbf{T}$ has different ones). The remainder of ViT is identical to the GPT case, with a [CLS] token concatenated along t — a sort-legal operation by Theorem 6.

6.3 U-Net

[RFB15], in our notation. Declare $s_1, s_2 : \mathbf{S}, c : \mathbf{C}, \tau : \mathbf{T}$ (diffusion time as a token-like axis [HJA20]), $\ell : \mathbf{L}$ (level/depth in the U).

A U-Net decoder block at level ℓ is

$$\text{UpBlock}_\ell(z, h_\ell) = \text{ResBlock}_\ell(\text{Concat}_c(\text{Upsample}_{s_1, s_2}(z), h_\ell)), \quad (14)$$

where h_ℓ is the skip-connected feature map from level ℓ of the encoder. The concatenation is along $c : \mathbf{C}$, which is sort-legal; concatenation along $\mathbf{B}$ would be flagged as a type error (the bug of “I concatenated the skip the wrong way” becomes unwriteable). The full denoiser is

$$\epsilon_\theta^{[b,c,s_1,s_2]}(x, \tau) = \text{Out} \circ \left(\bigodot_{\ell=L}^1 \text{UpBlock}_\ell \right) \circ \text{Mid} \circ \left(\bigodot_{\ell=1}^L \text{DownBlock}_\ell \right) \circ \text{In}(x + \phi(\tau)). \quad (15)$$

6.4 Mamba (selective state-space model)

[GD23] introduces a recurrent state that we model with sort $\mathbf{M}$. Declare $b : \mathbf{B}, t : \mathbf{T}, d : \mathbf{F}, m : \mathbf{M}$. The selective SSM update is

$$h^{[b,t,d,m]} = \overline{A}^{[b,t,d,m,m']} \cdot h^{[b,t-1,d,m']} + \overline{B}^{[b,t,d,m]} \cdot x^{[b,t,d]}, \quad (16)$$

$$y^{[b,t,d]} = C^{[b,t,d,m]} \cdot h^{[b,t,d,m]}, \quad (17)$$

where the state transition is a recurrence on $\mathbf{T}$ (legal because Theorem 2 admits ordered $\mathbf{T}$ -recurrence) and the contraction over m in the readout is licensed by sort $\mathbf{M}$. The selectivity — the dependence of $\overline{A}, \overline{B}, C$ on t — is visible in the signature; in S4 (the predecessor), these coefficients are t -independent and the t -axis is dropped from their signatures. The notation captures the difference at a glance. The recent state-space duality [DG24] can be expressed by exhibiting the structured-matrix decomposition of the $t \leftarrow t'$ contraction: $\mathfrak{A}$ and Mamba’s selective scan become two algebraic forms of the same typed operator.

6.5 Mixture of Experts

Declare $g : \mathbf{G}$ (expert axis) with $|g| = E$ experts. Following [Sha+17] and [FZS22], a MoE layer is

$$\text{MoE}\langle X^{[b,t,d]} \rangle = \text{Sum}_g(r^{[b,t,g]} \odot f_g^{[b,t,d]}(X^{[b,t,d]})), \quad (18)$$

where $r^{[b,t,g]} = \text{TopK}_g(\text{softmax}_g(R^{[b,t,d]}X^{[b,t,d]}))$ is the router and each f_g is a per-expert MLP indexed by g . The Sum_g reduction is licensed (sort $\mathbf{G}$ permits gated mixing); a Sum_b in this position would not be, and the typo would be caught.

The capacity-and-balance literature (auxiliary losses, expert capacity factors) becomes statements about *which fraction of the $\mathbf{G}$ -axis* receives nonzero router weight — a typed statement.

7 Open problems and proposed extensions

We list five directions in which the framework is incomplete and propose, for each, a concrete extension whose proof or implementation is left to future work.

7.1 Typed differentiation

The framework as developed handles forward semantics only. The reverse-mode derivative of a typed-named expression deserves its own calculus. The right starting point is the reverse derivative category framework [Coc+20], used by [Cru+22] and [Cru+24] to give a categorical semantics to backpropagation.

Proposal. Define a duality $(\cdot)^* : \mathcal{S} \rightarrow \mathcal{S}$ on sorts: $\mathbf{B}^* = \mathbf{B}$, $\mathbf{F}^* = \mathbf{F}$, $\mathbf{T}^* = \mathbf{T}$, $\mathbf{H}^* = \mathbf{H}$, $\mathbf{K}^* = \mathbf{K}$, $\mathbf{C}^* = \mathbf{C}$, $\mathbf{S}^* = \mathbf{S}$, $\mathbf{L}^* = \mathbf{L}$, $\mathbf{G}^* = \mathbf{G}$, $\mathbf{N}^* = \mathbf{N}$, $\mathbf{M}^* = \mathbf{M}$, and crucially $\mathbf{V}^* = \mathbf{F}$: the gradient of a vocabulary-axis logit lives in the feature space of the unembedding, not in the discrete vocabulary itself. For each typed named tensor $X^{[\Sigma]}$, the cotangent space T_X^* is the set of tensors with signature $\Sigma^* := \{(n_a, s_a^*, |a|) : a \in \Sigma\}$.

For each primitive operation O (Section 3.2) we specify its *typed adjoint* $O^\dagger$, which acts on cotangent tensors:

- **Contraction:** $(W^{[d,d']}X^{[b,t,d]})^\dagger$ on a cotangent $\bar{Y}^{[b,t,d']}$ is $(W^{[d,d']})^\top \bar{Y}^{[b,t,d']}$, which type-checks because $F^* = F$ and the contraction along d' is licensed.
- **Reduction:** $(\text{Sum}_a X)^\dagger$ on a cotangent $\bar{Y}^{[\Sigma \setminus a]}$ is $\text{Bcast}_a(\bar{Y})$, which type-checks because reduction–broadcast is the standard adjoint pair and $a^* = a$.
- **Pointwise:** $(\sigma\langle X \rangle)^\dagger$ on a cotangent $\bar{Y}$ is $\sigma'(X) \odot \bar{Y}$, the standard chain-rule term.
- **Attention:** $(\mathfrak{A}_{k,v}^{h;t \leftarrow t'}[Q, K, V])^\dagger$ produces three cotangent tensors $\bar{Q}, \bar{K}, \bar{V}$ with the same sort signatures as Q, K, V themselves, by the well-known closed-form gradient of softmax-attention.

Conjecture 24 (Typed adjoint preserves sort correctness). *For every sort-correct forward expression E built from the primitives of Section 3, the reverse-mode adjoint $E^\dagger$ defined by composition of primitive adjoints is sort-correct under the dual sort assignment.*

A proof would establish that each primitive’s adjoint is sort-correct (case analysis on a finite set of operators) and that adjoint composition respects sort discipline (a categorical argument analogous to [FST19]). The non-trivial cases are non-standard primitives like attention and Mamba’s selective scan; for both, we conjecture sort-correctness follows from the existence of a closed-form gradient.

Type-discipline implications. With Theorem 24 in hand, two design patterns gain typed semantics:

Gradient stops. A StopGrad operator becomes a typed coercion $\Sigma \rightarrow \Sigma^*$ that is the identity in the forward pass and the zero map in the backward pass. The asymmetry is visible in the signature: StopGrad has different forward and adjoint actions, which the type system tracks.

Straight-through estimators. [BLC13] A Quantize $_{p \rightarrow p'}$ operator that quantizes a tensor’s precision in the forward pass is paired with an adjoint that is the identity (passing the gradient straight through). The framework of Section 7.5 extends this to a precise typed account: STE is the canonical adjoint of a precision-changing operator under the rule that gradient flow remains in the higher precision.

7.2 Stochastic axes

Dropout [Sri+14], sampling, and the noise axis in diffusion all warrant first-class treatment. We introduced sort $\mathbf{N}$ in Section 2, with the rule that $\mathbf{N}$ -axes may not be reduced before the loss-time marginalization $\mathbb{E}_n$.

Proposal. Use the refinement machinery of Section 2.5 to build a hierarchy of stochastic structure inside $\mathbf{N}$:

- $\mathbf{N}^{\text{iid}} \sqsubseteq \mathbf{N}$: i.i.d. noise samples ($G_{\mathbf{N}^{\text{iid}}} = S_n$). Admits Monte Carlo marginalization $\mathbb{E}_n[X^{\mathbf{N}^{\text{iid}}}] \approx \frac{1}{n} \sum_n X$ with the standard $1/\sqrt{n}$ error rate. The dropout setting.
- $\mathbf{N}^{\text{joint}} \sqsubseteq \mathbf{N}$: jointly distributed samples (e.g., a Gaussian process draw at multiple positions). Marginalization is licensed only after the joint structure is integrated out via an explicit Cov-based reduction.
- $\mathbf{N}^\pi \sqsubseteq \mathbf{N}$: samples conditioned on a policy π [Wil92; SB18]. Importance-sampling reweighting becomes a typed sort-changing operator $\mathbf{N}^\pi \rightarrow \mathbf{N}^{\pi'}$ that re-weights samples by $\pi'(\cdot)/\pi(\cdot)$.

Process-indexed stochastic axes. For diffusion [HJA20] and other stochastic-process settings, we further allow noise axes parameterized by a process index. Define a *process lift* $\Pi : \mathbb{T} \rightarrow \mathbb{N}$ that maps a time axis to a time-indexed family of noise axes; the lifted sort is $\mathbb{N}_t$, with the rule that two indices $t_1 \neq t_2$ along $\mathbb{T}$ produce *independent* (or correlated, per the process specification) draws. The forward diffusion process $X_t = \alpha_t X_0 + \sigma_t \epsilon_t$ with $\epsilon_t \sim \mathcal{N}(0, I)$ becomes the typed expression

$$X^{[b,c,s_1,s_2,t]} = \alpha^{[t]} \odot X_0^{[b,c,s_1,s_2]} + \sigma^{[t]} \odot \epsilon^{[b,c,s_1,s_2,t,n_t]},$$

where $n_t : \mathbb{N}_t$ is the process-indexed noise axis. The reverse process is then a typed operator that consumes one n_t -slice per time step.

Typed estimators. The proposal makes the choice of statistical estimator visible at the type level: SGD operates on $\mathbb{B}^{\text{iid}}$ minibatches, variational inference [KW14] operates on parametric $\mathbb{N}$ -axes, diffusion operates on process-indexed $\mathbb{N}_t$ -axes, and policy gradients operate on $\mathbb{N}^\pi$ -axes. The estimator’s variance properties become typed lemmas: e.g., reparameterization is the typed claim that a $\mathbb{N}^\pi$ -axis can be re-expressed as a deterministic transform of an $\mathbb{N}^{\text{iid}}$ -axis when the parameterization permits.

7.3 Dependent shape types

Variable sequence length is currently a footnote. In practice $|t|$ is not a constant but depends on the batch element: $|t| = \ell_b$ for some $\ell_b \in \mathbb{N}^{[b]}$. Honest support for this requires dependent types. The Dex language [Pas+21] demonstrates that dependent shape types are workable for ML; the question is whether they admit ergonomic notation.

Proposal: a tractable fragment. Rather than full dependent types, we propose a restricted fragment with three constructions, each arising directly from common ML patterns:

- (1) *Length-indexed sequences.* An axis $t : \mathbb{T}_{\leq T_{\max}}$ with an associated length variable $\ell : \mathbb{N}^{[b]}$. Operations that respect length (causal masking, packing, cumulative sums up to ℓ_b) are legal; operations that ignore it (full attention beyond ℓ_b) require explicit unmasking via $\text{Slice}_{[0,\ell_b)}$ or a declared mask predicate.
- (2) *Ragged tensors.* A pair (X, ℓ) where $X^{[b,t,d]}$ is a padded tensor and $\ell^{[b]}$ is a per-batch length. Reductions along t become offset-aware: $\text{Mean}_t X = (\sum_{i=0}^{\ell_b-1} X_i) / \ell_b$ rather than a uniform $T_{\max}$ -average. The framework declares which reductions automatically respect length and which require explicit length-awareness.
- (3) *Structured masks.* An axis annotated with a mask predicate $m^{[a,a']}$. Padded sequences become $t : \mathbb{T}$ with mask $\mathbb{K}[i < \ell_b]$; sliding-window attention becomes $t : \mathbb{T}$ with mask $\mathbb{K}[|i-j| < w]$; block-sparse attention becomes a block-predicate mask. The mask predicate is a typed object with signature $[a, a']$, so attention masks are first-class typed tensors and not informal annotations on the diagonals.

Proposition 25 (Decidability of the dependent fragment). *Type checking under constructions (1)–(3) remains decidable in time linear in the size of the expression tree, provided the dependency takes the form of first-order arithmetic propositions over axis lengths and indices.*

Proof sketch. The dependent fragment introduces propositional constraints of the form $\ell_b \leq T_{\max}$ and $i < \ell_b$, both linear arithmetic. Type checking reduces to a series of such constraints, each verifiable in constant time. Decidability follows from Presburger arithmetic. $\square$

Corollary 26 (Interpretability tools work on padded batches). *Within the dependent fragment, the interpretability corollaries (Theorems 21 to 23) extend to batches with variable sequence*

length: the logit lens, linear probes, and activation patching are well-typed at every layer for every batch element, regardless of padding.

This is the typed expression of the (currently informal) practitioner observation that interpretability tools should pad-mask-aware. The corollary makes the well-typedness explicit and forecloses the bug class of “patched activations leaked into padded positions” that several interpretability libraries have hit.

7.4 Hardware and distribution sorts

Tensor-parallel [Sho+19], ZeRO-style [Raj+20], and FSDP [Zha+23] distributed training introduce a sharding dimension orthogonal to the semantic axes.

Proposal: D as a meta-sort. Introduce a sort $\mathbf{D}$ for device. Treat $\mathbf{D}$ as a *meta-sort* that sits alongside the primary sorts: every tensor in a distributed context carries, in addition to its semantic signature, a *sharding annotation* $\text{shard} : \Sigma \rightarrow (\mathcal{P}(\mathbf{D}))^{|\Sigma|}$ specifying, for each primary axis a , how its indices are partitioned across devices. Concretely $\text{shard}_a : [|a|] \rightarrow [|d|]$ assigns each index of a to a device.

Distributed collective operations become typed reductions and broadcasts on $\mathbf{D}$:

- All-reduce: $\text{Sum}_d X^{[\Sigma, d]}$ (a reduction along $\mathbf{D}$ into a $\mathbf{D}$ -replicated tensor).
- All-gather: Concat_a along a sharded axis a , with d as the partition.
- Reduce-scatter: a composition of all-reduce with a re-sharding to a different shard_a .
- Pipeline parallelism: a stack $\bigodot_\ell f_\ell$ where each f_ℓ has a different $\mathbf{D}$ -shard, communicating via $\mathbf{D}$ -typed activations passed across pipeline stages.

Sharding correctness theorem. The proposal makes sharding correctness an equivariance condition.

Conjecture 27 (Sharding correctness). *A distributed implementation of a forward pass E is computationally equivalent to its single-device counterpart iff the sharding annotation shard commutes with every operation in E in the sense that, for every primary axis a touched by an operation O in E , the action of O on a is G_{s_a} -equivariant under the device permutations induced by shard_a (Theorem 7).*

If proved, the conjecture immediately implies the standard rules collected by Megatron-LM and its successors, now derivable rather than empirical:

- Sharding a $\mathbf{B}$ -axis is always legal (data parallelism trivially preserves $G_{\mathbf{B}} = S_n$ exchangeability).
- Sharding a $\mathbf{F}$ -axis requires *matched sharding* of every weight that contracts with f (tensor parallelism with paired weight sharding); without the matching, contraction is not equivariant.
- Sharding a $\mathbf{T}$ -axis under causal masking requires causal-respecting communication (one cannot freely re-shuffle tokens across devices).
- Sharding an $\mathbf{H}$ -axis is always legal (head parallelism — heads are exchangeable, $G_{\mathbf{H}} = S_n$).
- Sharding an $\mathbf{M}$ -axis across recurrence steps is illegal: the recurrent state must be local to its temporal index, since $G_{\mathbf{M}} = \{e\}$ (Theorem 7).

This formalizes parallelization rules currently passed down as folklore and gives a single notation that expresses both the algorithm and the parallelization strategy. ZeRO and FSDP variants become different choices of shard on weight tensors; mixture-of-experts model parallelism becomes a shard_g on the expert axis $\mathbf{G}$.

7.5 Quantization and sub-axis sorts

Proposal: precision as a refinement. Treat numerical precision as a refinement on every sort. Each axis $a : s$ acquires a precision tag $p \in \mathcal{P}$, where $\mathcal{P}$ is a small lattice of precisions ordered by representational power:

$$\text{int4} < \text{int8} < \text{fp4} < \text{fp8} < \text{fp16} < \text{bf16} < \text{fp32} < \text{fp64}.$$

The refinement is annotated as a colon-separated tag: $X^{[d:F;\text{bf16}]}$ denotes a feature-axis tensor at **bf16** precision. The precision tag participates in the sort algebra: an operation O between operands at precisions p_1, p_2 produces output at $\text{lub}(p_1, p_2)$ unless an explicit cast is given.

Mixed-precision rules. The proposal codifies the well-known empirical rules of mixed-precision training [Mic+18]:

- *Contractions* allow precision lowering on the operands but require an accumulator at $\geq \text{fp32}$. The dot product of two **bf16** tensors is a typed expression $X^{[\dots;\text{bf16}]} \cdot Y^{[\dots;\text{bf16}]}$ producing an **fp32**-precision output unless an explicit cast follows.
- *Reductions* enforce a minimum accumulator precision: $\text{Sum}_a X^{[\Sigma;\text{fp16}]}$ is a type error unless an explicit accumulator is declared (e.g., $\text{Sum}_a^{\text{fp32}}$). This catches the well-documented bug class of summing **fp16** values without an **fp32** accumulator.
- *Quantization and dequantization* are explicit sort-changing operators $\text{Quantize}_{p \rightarrow p'}$ and $\text{Dequantize}_{p' \rightarrow p}$ for $p' < p$. These are paired with associated scale tensors at higher precision: a **int8**-quantized weight is the typed pair $(W^{[d,d';\text{int8}]}, s^{[d';\text{fp16}]})$, where s is the per-channel scale. Per-channel quantization [Jac+18] and group-wise quantization (e.g., GPTQ, [Fra+23]) are different choices of which axis the scale lives on.

Quantization-aware training. A typed framework for QAT emerges naturally. The forward pass is at low precision; the gradient flow is at high precision. Specifically, for a quantize operator $\text{Quantize}_{\text{fp32} \rightarrow \text{int8}}$, the typed adjoint of Section 7.1 is the *straight-through estimator* [BLC13]: the identity on **fp32** cotangents. The asymmetry between forward and adjoint precisions is visible in the type signature, and the canonical STE is recovered as the only adjoint that is type-correct without introducing a second precision-changing operator.

Remark 28 (Predicted bug class caught). The *accumulator-overflow* bug, in which a long sum of **fp16** values silently saturates, becomes a type error: Sum_a on a tensor at precision $\leq \text{fp16}$ is rejected without an explicit accumulator declaration. This formalizes the empirical rule that long reductions must be performed in higher precision [Mic+18] and gives a typed account of the practice of “Kahan summation,” “compensated reductions,” and similar accumulator-precision tricks.

Closing remark

The thesis of this paper is prescriptive: we have offered a type discipline for deep learning, not merely a notation, and the test of the framework is the falsifiable criterion of Section 1 — whether common axis-confusion bugs become syntax errors. Section 3.4 discharges that test on a documented bug from the contrastive-learning literature, in which BatchNorm applied to a paired-sample batch is an information leak that the type checker rejects, with all three canonical fixes (LayerNorm, ShuffleBN, view-splitting) recoverable as discharges of the constraint. We do not claim that all deep learning bugs are sort errors; we claim that a substantial and well-documented class of them are, and that this class is now type-checkable.

The relation to Einstein’s notation is precise: not a compression of writing, but a discipline that promotes meaningful coincidences from the semantic level to the syntactic. Such a

framework does not just describe architectures; it suggests which architectures are *typeable* and therefore natural, and which require sort gymnastics that signal something is off. It also offers a prediction: if the framework is right, the rate of axis-confusion bugs in deep learning code should drop sharply once the discipline is internalized, just as the rate of metric-signature bugs in physics dropped once Einstein notation was internalized.

We have given the skeleton: a sort algebra with refinement (Sections 2 and 2.5), a typed named-tensor calculus with a worked bug-catching example (Sections 3 and 3.4), a canonical attention glyph (Section 4), residual and stack combinators with a residual-stream invariance theorem (Section 5), case studies on five architectures (Section 6), and *proposed extensions* for the five open problems of Section 7: typed differentiation via reverse derivative categories with a dual sort assignment (Section 7.1); a refinement hierarchy of stochastic axes covering dropout, variational inference, diffusion, and policy gradients (Section 7.2); a tractable fragment of dependent shape types covering length-indexed sequences, ragged tensors, and structured masks (Section 7.3); a meta-sort D for distributed training with a sharding-correctness conjecture (Section 7.4); and a precision refinement giving a typed account of mixed precision and quantization-aware training (Section 7.5). What remains is the work of proving the conjectures (Theorems 24 and 27) and instantiating the framework as a working type checker against which the empirical claim of Section 1, criterion (b), can be measured.

Acknowledgments

The author thanks the Anthropic interpretability team for the framing of the residual stream as a communication channel [Elh+21], which provided the central guiding metaphor for Section 5, and the named-tensor working group of [CRB23] for the substrate on which this paper is built.

A Proof of Conjecture 24

We prove that the typed adjoint preserves sort correctness, promoting Conjecture 24 of Section 7.1 to a theorem. The proof follows the standard pattern for compositional type-soundness results: a finite case analysis on the primitive operations, plus a closure-under-composition argument that mirrors the categorical-ML lineage of [FST19] and [Cru+24].

A.1 Setup

Recall from Section 7.1 the duality $(\cdot)^* : \mathcal{S} \rightarrow \mathcal{S}$, which fixes every sort except $V^* = F$. For a tensor signature $\Sigma = \{(n_a, s_a, |a|)\}$, the dual signature is $\Sigma^* = \{(n_a, s_a^*, |a|)\}$, and the cotangent space T_X^* is the set of typed named tensors with signature Σ^* .

A *primitive operation* is one of: contraction, reduction, pointwise, broadcast, concatenation, normalization, or attention. For each primitive $O : X^{[\Sigma_{\text{in}}]} \mapsto Y^{[\Sigma_{\text{out}}]}$, the *typed adjoint* $O^\dagger : \overline{Y}^{[\Sigma_{\text{out}}^*]} \mapsto \overline{X}^{[\Sigma_{\text{in}}^*]}$ acts on cotangent tensors and is defined by the standard reverse-mode rules.

Definition 29 (Sort correctness). An expression E is *sort-correct* if every primitive node in its expression tree is a legal application under Theorems 1 to 7 and the legality table of Section 2.4.

Theorem 30 (Typed adjoint preserves sort correctness). *For every sort-correct forward expression E built from the primitives of Section 3, the reverse-mode adjoint $E^\dagger$ defined by composition of primitive adjoints is sort-correct under the dual sort assignment.*

The proof has three components: a primitive-by-primitive verification (Section A.2), a composition lemma (Section A.3), and a check that the duality $(\cdot)^*$ is involutive in the sense needed for the final assembly (Section A.4).

A.2 Per-primitive verification

We show that each primitive’s adjoint is sort-correct.

Lemma 31 (Contraction adjoint). *Let E be the contraction $E = X \cdot Y$ with $X^{[\Sigma_X]}, Y^{[\Sigma_Y]}$, and shared axes $A = \Sigma_X \cap \Sigma_Y$ partitioned into A_{contr} (those whose sorts license contraction in E ’s context) and A_{align} (those that pass through). The adjoints with respect to X and Y are*

$$\begin{aligned} E_X^\dagger(\bar{E}) &= \bar{E} \cdot Y, \\ E_Y^\dagger(\bar{E}) &= X \cdot \bar{E}, \end{aligned}$$

where $\bar{E}$ has signature $(\Sigma_X \cup \Sigma_Y) \setminus A_{\text{contr}}$ dualized. Both adjoint contractions are sort-correct in E ’s context.

Proof. The contraction $\bar{E} \cdot Y$ shares with $\bar{E}$ all axes of $\Sigma_Y \setminus A_{\text{contr}}$ (which were aligned or only- Y in the forward pass) and all axes of A_{contr} (which appear in $\bar{E}$ via Y ’s side). The dualized sorts agree because $(\cdot)^*$ fixes every contractable sort: $F^* = F$, $K^* = K$, $C^* = C$, $T^* = T$, $H^* = H$, $S^* = S$, $M^* = M$, $G^* = G$. The licensing context is preserved: the adjoint of a contraction-in-attention is itself a contraction-in-attention. The argument for $E_Y^\dagger$ is symmetric.

The only subtle case is $V^* = F$: a contraction along V does not occur in any forward primitive (Axiom 3 forbids it), so this case is vacuous in the contraction primitive. $\square$

Remark 32 (Implementation discovery). The proof of Theorem 31 relies on the partition $A = A_{\text{contr}} \sqcup A_{\text{align}}$, which the prose of Section 3.2 implicitly conflates. Implementing the type checker (Section B) revealed that this distinction is load-bearing: shared axes whose sorts always pass through (sort B, H, G, N, L) must be treated as alignment axes, never as contraction candidates, regardless of operator context. The implementation enforces this with a fixed set $\text{ALWAYS_ALIGN} \subset \mathcal{S}$.

Lemma 33 (Reduction adjoint). *For a reduction $E = \text{Sum}_a X$ (or Mean_a , or any linear reduction), the adjoint is*

$$E^\dagger(\bar{E}) = \text{Bcast}_a(\bar{E}),$$

which is sort-correct iff a admits broadcast under the dual sort. Broadcast and reduction are the standard adjoint pair.

Proof. E is sort-correct only if a ’s sort licenses the reduction (Axioms 1 and 4). The adjoint introduces an axis of the same name and sort. Since the duality fixes all primitive sorts and broadcast is licensed for every sort that admits reduction in our table (with the exception of V , where the adjoint of a LogSumExp produces a softmax rather than a uniform broadcast — still broadcast-shaped, still sort-correct), the adjoint type-checks.

For the N -axis case (only legal reduction is $\mathbb{E}_n$ at loss time), the adjoint is the reparameterized broadcast, which is sort-correct by the same argument. $\square$

Lemma 34 (Pointwise adjoint). *For $E = \sigma\langle X \rangle$ with smooth σ , the adjoint is*

$$E^\dagger(\bar{E}) = \sigma'(X) \odot \bar{E},$$

which preserves the signature exactly. Sort-correct trivially.

Lemma 35 (Concatenation adjoint). *For $E = \text{Concat}_a(X, Y)$, the adjoint splits along a :*

$$E^\dagger(\bar{E}) = (\text{Slice}_{a,[0,|a|_X]}(\bar{E}), \text{Slice}_{a,[|a|_X,|a|_X+|a|_Y]}(\bar{E})).$$

Slice is licensed on every sort that admits Concat (Axiom 6 and the legality table), and the dualized sort matches by fixedness of $(\cdot)^$ on all concatenable sorts.*

Lemma 36 (Normalization adjoint). *For $E = \text{Norm}_a\langle X \rangle$, the adjoint is the standard Layer-Norm gradient, which involves a reduction along a followed by a broadcast back along a . Both substeps are sort-correct on a 's sort by the legality table (since Norm_a being legal already requires those reductions to be legal).*

Lemma 37 (Attention adjoint). *For $E = \mathfrak{A}_{k,v}^{h;t \leftarrow t'}[Q, K, V \mid M]$, the adjoint with respect to each of Q, K, V is the well-known closed-form softmax-attention gradient, which factors through three further attention-context contractions on the same sort signatures $[h, t, t', k, v]$. Sort-correctness follows from Theorem 31 applied three times.*

Proof. The closed-form softmax-attention gradient (see, e.g., [Vas+17; Dao+22]) decomposes into:

$$\begin{aligned}\bar{V} &= \text{softmax}_{t'}(s)^\top \cdot \bar{E}, \\ \bar{s} &= V \cdot \bar{E}^\top - \langle V \cdot \bar{E}^\top, \text{softmax}_{t'}(s) \rangle \cdot \text{softmax}_{t'}(s), \\ \bar{Q} &= \frac{1}{\sqrt{|k|}} \bar{s} \cdot K, \quad \bar{K} = \frac{1}{\sqrt{|k|}} \bar{s} \cdot Q,\end{aligned}$$

where s is the pre-softmax score tensor. Each of $\bar{V}, \bar{Q}, \bar{K}$ is a contraction in the attention context, with sort signatures matching V, Q, K respectively. By Theorem 31, each is sort-correct. $\square$

A.3 Composition closure

Lemma 38 (Adjoint composition is sort-correct). *If $E_1 : \Sigma_0 \rightarrow \Sigma_1$ and $E_2 : \Sigma_1 \rightarrow \Sigma_2$ are sort-correct primitives, and their adjoints $E_1^\dagger, E_2^\dagger$ are sort-correct (by Section A.2), then the composite adjoint*

$$(E_2 \circ E_1)^\dagger = E_1^\dagger \circ E_2^\dagger$$

is sort-correct.

Proof. By the chain rule for reverse-mode AD, $(E_2 \circ E_1)^\dagger = E_1^\dagger \circ E_2^\dagger$ as functions on cotangents. The composition is sort-correct iff (i) the output signature of $E_2^\dagger$ matches the input signature of $E_1^\dagger$ and (ii) each component is sort-correct in isolation.

For (i): $E_2^\dagger$ produces a cotangent of signature Σ_1^* , which is exactly the input signature required by $E_1^\dagger$. For (ii): both components are sort-correct by hypothesis.

The argument extends inductively to expression trees with branching (parameter-shared subexpressions) by adding cotangents at the merge points, which is also sort-preserving since sorts are closed under $+$. $\square$

A.4 Involution and final assembly

Lemma 39 (Duality is involutive on used sorts). *The duality $(\cdot)^* : \mathcal{S} \rightarrow \mathcal{S}$ satisfies $s^{**} = s$ for every sort s except $\mathbf{V}$, and for $\mathbf{V}$ it satisfies $\mathbf{V}^{**} = \mathbf{V}^* = \mathbf{F}$ and $\mathbf{F}^* = \mathbf{F}$ — i.e., it stabilizes to $\mathbf{F}$ after one application. This is consistent because $\mathbf{V}$ -axes appear only as outputs of an unembedding in a forward expression, never as intermediate representations, so the only $\mathbf{V}^*$ axis that ever arises in $E^\dagger$ is the dualized output, which lies in $\mathbf{F}$ where the duality is stable.*

Proof of Theorem 30. By induction on the expression-tree depth of E . For depth-1 expressions (a single primitive), the result follows from Theorems 31, 33 and 37 and the analogous lemmas for the remaining primitives. For depth- n expressions, decompose $E = E_n \circ E_{<n}$ where $E_{<n}$ is the depth- $(n-1)$ subexpression. The inductive hypothesis gives sort-correctness of $E_{<n}^\dagger$; Theorem 38 extends it to $E^\dagger = E_{<n}^\dagger \circ E_n^\dagger$. $\square$

Corollary 40 (Gradient stops are typed coercions). *The StopGrad operator, which is the identity in the forward pass and the zero map in the backward pass, is the unique typed operator whose forward signature is $\Sigma \rightarrow \Sigma$ and whose adjoint signature is $\Sigma^* \rightarrow 0$. In particular it is sort-correct as an asymmetric coercion.*

Corollary 41 (Straight-through estimators are typed adjoints). *For a quantization operator $\text{Quantize}_{p \rightarrow p'} : X^{[\Sigma; p]} \mapsto Y^{[\Sigma; p']}$ with $p' < p$ in the precision lattice (Section 7.5), the straight-through estimator [BLC13] — the identity adjoint $\text{Quantize}^\dagger(\bar{Y}) = \bar{Y}$ at precision p — is the unique sort-correct adjoint that does not introduce a second precision-changing operator in the backward pass.*

Proof. Any adjoint of $\text{Quantize}_{p \rightarrow p'}$ produces a cotangent at the input precision p from a cotangent at the output precision p' . The identity is the unique map between p' and p that does not require a second precision-changing operator in its definition. Hence STE is the canonical typed adjoint. $\square$

This proves Conjectures 24 and lifts the design patterns of Section 7.1 (gradient stops, straight-through estimators) to typed corollaries.

B Reference implementation

A reference implementation of the framework, comprising approximately 530 lines of Python wrapping NumPy, is available alongside this paper. The implementation provides:

- the sort algebra of Section 2 as a Python enum and the refinement order of Section 2.5 as a frozen dataclass;
- the typed named tensor of Section 3 as a class wrapping a NumPy array with a tuple of sorted axes;
- primitive operations `contract`, `reduce`, `normalize`, `concat`, `attention`, each performing the type-checking algorithm of Theorem 16 before delegating to NumPy;
- a test suite of 13 cases including the four-step worked example of Section 3.4: the buggy expression rejected at type-checking time, and each of the three canonical fixes (LayerNorm, ShuffleBN, view-splitting) accepted with the expected typed signature.

The implementation served two purposes beyond demonstration. First, it confirmed that the type-checking algorithm is decidable in practice and runs in negligible time (< 1 ms per check). Second, it surfaced one design subtlety that the prose of Section 3.2 had elided: the partition of shared axes into *contraction* and *alignment* axes (some sorts — **B**, **H**, **G**, **N**, **L** — always align when shared, regardless of operator context). This refinement to the contraction rule is incorporated into Theorem 31 and the implementation is the canonical reference for the rule.

References

- [Ain+23] Joshua Ainslie et al. “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2023. arXiv: [2305.13245](#).
- [BB17] Jacob Biamonte and Ville Bergholm. *Tensor Networks in a Nutshell*. 2017. arXiv: [1708.00006](#).
- [BKH16] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. *Layer Normalization*. 2016. arXiv: [1607.06450](#).

- [BLC13] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation*. 2013. arXiv: [1308.3432](#).
- [Bri+23] Trenton Bricken et al. “Towards Monosemanticity: Decomposing Language Models with Dictionary Learning”. In: *Transformer Circuits Thread* (2023). URL: <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- [Car+20] Mathilde Caron et al. “Unsupervised Learning of Visual Features by Contrasting Cluster Assignments”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020.
- [Che+20] Ting Chen et al. “A Simple Framework for Contrastive Learning of Visual Representations”. In: *International Conference on Machine Learning (ICML)*. 2020.
- [Coc+20] Robin Cockett et al. “Reverse Derivative Categories”. In: *28th EACSL Annual Conference on Computer Science Logic (CSL 2020)*. 2020. arXiv: [1910.07065](#).
- [Con+23] Arthur Conmy et al. *Towards Automated Circuit Discovery for Mechanistic Interpretability*. 2023.
- [CRB23] David Chiang, Alexander M. Rush, and Boaz Barak. “Named Tensor Notation”. In: *Transactions on Machine Learning Research* (2023). arXiv: [2102.13196](#). URL: <https://openreview.net/forum?id=hVT7SHlilx>.
- [Cru+22] Geoffrey S. H. Cruttwell et al. “Categorical Foundations of Gradient-Based Learning”. In: *European Symposium on Programming (ESOP)*. Springer, 2022, pp. 1–28.
- [Cru+24] Geoffrey S. H. Cruttwell et al. *Deep Learning with Parametric Lenses*. 2024. arXiv: [2404.00408](#).
- [Dao+22] Tri Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022.
- [Dee24] DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. 2024. arXiv: [2405.04434](#).
- [DG24] Tri Dao and Albert Gu. “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality”. In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 10041–10071.
- [Dos+21] Alexey Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *International Conference on Learning Representations (ICLR)*. 2021.
- [Elh+21] Nelson Elhage et al. “A Mathematical Framework for Transformer Circuits”. In: *Transformer Circuits Thread* (2021). URL: <https://transformer-circuits.pub/2021/framework/index.html>.
- [Fra+23] Elias Frantar et al. “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers”. In: *International Conference on Learning Representations (ICLR)*. 2023.
- [FST19] Brendan Fong, David I. Spivak, and Rémy Tuyéras. “Backprop as Functor: A Compositional Perspective on Supervised Learning”. In: *2019 34th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. IEEE, 2019, pp. 1–13.
- [FTC20] Giovanni de Felice, Alexis Toumi, and Bob Coecke. *DisCoPy: Monoidal Categories in Python*. 2020. arXiv: [2005.02975](#).

- [FZS22] William Fedus, Barret Zoph, and Noam Shazeer. “Switch Transformer: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity”. In: *Journal of Machine Learning Research* 23.120 (2022), pp. 1–39.
- [GD23] Albert Gu and Tri Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2023. arXiv: [2312.00752](https://arxiv.org/abs/2312.00752).
- [He+16] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.
- [He+20] Kaiming He et al. “Momentum Contrast for Unsupervised Visual Representation Learning”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 9729–9738.
- [HH17] Stephan Hoyer and Joseph J. Hamman. “xarray: N-D Labeled Arrays and Datasets in Python”. In: *Journal of Open Research Software* 5.1 (2017).
- [HJA20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. “Denoising Diffusion Probabilistic Models”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 6840–6851.
- [Hua+16] Gao Huang et al. “Deep Networks with Stochastic Depth”. In: *European Conference on Computer Vision (ECCV)*. Springer, 2016, pp. 646–661.
- [IS15] Sergey Ioffe and Christian Szegedy. “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”. In: *International Conference on Machine Learning (ICML)*. 2015.
- [Jac+18] Benoit Jacob et al. “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 2704–2713.
- [Kat+20] Angelos Katharopoulos et al. “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention”. In: *International Conference on Machine Learning (ICML)*. 2020.
- [KW14] Diederik P. Kingma and Max Welling. “Auto-Encoding Variational Bayes”. In: *International Conference on Learning Representations (ICLR)*. 2014.
- [Lan+20] Zhenzhong Lan et al. *ALBERT: A Lite BERT for Self-supervised Learning of Language Representations*. 2020.
- [Men+22] Kevin Meng et al. “Locating and Editing Factual Associations in GPT”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022.
- [Mic+18] Paulius Micikevicius et al. “Mixed Precision Training”. In: *International Conference on Learning Representations (ICLR)*. 2018.
- [nos20] nostalgebraist. *Interpreting GPT: The Logit Lens*. LessWrong. 2020. URL: <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>.
- [Ols+22] Catherine Olsson et al. “In-context Learning and Induction Heads”. In: *Transformer Circuits Thread* (2022). URL: <https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html>.
- [Pas+21] Adam Paszke et al. “Getting to the Point: Index Sets and Parallelism-Preserving Autodiff for Pointful Array Programming”. In: *Proceedings of the ACM on Programming Languages* 5.ICFP (2021), pp. 1–29. DOI: [10.1145/3473593](https://doi.org/10.1145/3473593).
- [Raj+20] Samyam Rajbhandari et al. “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models”. In: *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–16.

- [RFB15] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. In: *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. Springer, 2015, pp. 234–241.
- [Rog22] Alex Rogozhnikov. *Einops: Clear and Reliable Tensor Manipulations with Einstein-like Notation*. 2022. URL: <https://openreview.net/forum?id=oapKSVM2bcj>.
- [SB18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. MIT Press, 2018.
- [Sha+17] Noam Shazeer et al. “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”. In: *International Conference on Learning Representations (ICLR)*. 2017.
- [Sha19] Noam Shazeer. *Fast Transformer Decoding: One Write-Head is All You Need*. 2019. arXiv: [1911.02150](https://arxiv.org/abs/1911.02150).
- [Sho+19] Mohammad Shoeybi et al. *Megatron-LM: Training Multi-billion Parameter Language Models Using Model Parallelism*. 2019. arXiv: [1909.08053](https://arxiv.org/abs/1909.08053).
- [Sri+14] Nitish Srivastava et al. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15 (2014), pp. 1929–1958.
- [SS16] Edwin Stoudenmire and David J. Schwab. “Supervised Learning with Tensor Networks”. In: *Advances in Neural Information Processing Systems* 29 (2016).
- [Su+24] Jianlin Su et al. “RoFormer: Enhanced Transformer with Rotary Position Embedding”. In: *Neurocomputing* 568 (2024), p. 127063. arXiv: [2104.09864](https://arxiv.org/abs/2104.09864).
- [Vas+17] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.
- [Wil92] Ronald J. Williams. “Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning”. In: *Machine Learning* 8 (1992), pp. 229–256.
- [Zha+23] Yanli Zhao et al. *PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel*. 2023. arXiv: [2304.11277](https://arxiv.org/abs/2304.11277).
- [ZS19] Biao Zhang and Rico Sennrich. *Root Mean Square Layer Normalization*. 2019.